⇒ Random Forest (RF)

↳ Provides an improvement over bagged trees by decorrelates the trees.

As in bagging, we build a no. of DTs on bootstrapped samples but while building DTs, each time a split in a tree is considered a random sample of m predictors is chosen as split from full set of p predictors. A fresh sample of m predictors is taken at each split & typically we take $m \approx \sqrt{p}$. If we take some predictors for each DT then it creates correlation, hence predictors from bagged trees will be highly correlated & avging many highly correlated quantities don't lead to as large of a reduction in variance as avging many uncorrelated quantities., RF overcome it.

↳ As with bagging, RF will not overfit if we increase B, so in practice, we use a value of B sufficiently large. for settles error rate.

⇒ Boosting

↳ Trees are grown sequentially. Each tree is grown using info^n from previous grown tree. Boosting does not involve bootstrap sampling; instead each tree is fit on a modified version of original data set.

↳ Boosting learns slowly. Given the current model, we fit a DT to the residuals from the model, i.e. we fit a tree using the current residuals rather than outcome Y as the response. We then add this to new DT & update residuals.

↳ By fitting small trees to residuals, we slowly improve $\hat{f}$ in areas where it does not perform well.

↳ In general, statistical learning approaches that learn slowly ~~tend~~ tend to perform well.

↳ Boosting has 3 tuning parameters → # of trees (overfit if B is large)
↳ shrinkag parameter λ.
↳ # of splits.

↳ It uses a weighted sum of trees.

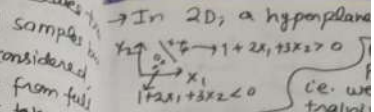
⇒ Support Vector Machines (SVM) (in 1990s by V. Vapnik).

↳ Generalizat^n of Maximum margin Classifier. (Classes are separated by linear bound)

↳ SVM (extention of SVC (support vector classifier) accomodate non-linear class bound)

→ Hyperplane is a flat affine (subspace need not pass through origin) subspace of dimension p-1. For ex. In 2D, hyperplane is a flat 1-D subspace = line

→ In 2D, a hyperplane



i.e. we trained

the smallest such distan

& is known as margin

→ When p is large

→ Support vectors "sup

sense that if these po

hyperplane would mo

→ SVC = Soft margin C

→ C (hyperparameter) =

→ $\underset{\beta_0, \beta_1, \ldots \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^{n} \right.$

λ = high ⇒ $\beta_1, \beta_2 \ldots$ sm

→ $L(x, y, \beta) = \sum_{i=1}^{n}$

⊗ Hypothesis Testing =

→ p-values are used te

→ Ex. Linear Relat'ship β=

→ 1st we define Null &

that summarizes the

compute p-value that

comparable or more

finally based on p-val

→ World → H_0 = default + s

H_a = Something

→ Test statistic 'T' summ

our data are consistent

→ p-value is defined as

a test statistic equal to

→ Smaall p-value provi

→ The dist^n of test-statisti

Decision ↓	H_0	H_a	Truth →
Reject H_0	Type I error	Correct (Power)	
Don't Reject	Correct	Type-II error	

⇒ Random Forest (RF)

↳ Provides an improvement over bagged trees by decorrelates
As in bagging, we build a no. of DTs on bootstrapped samples
while building DTs, each time a split in a tree is considered
a random sample of m predictors is chosen as split from full
set of p predictors. A fresh sample of m predictors is taken at
each split & typically we take $m \approx \sqrt{p}$. If we take same predictors
for each DT then it creates correlation, hence predictors from bagged
trees will be highly correlated & averaging many highly correlated
quantities don't lead to as large of a reduction in variance as
averaging many uncorrelated quantities. RF overcome it.

↳ As with bagging, RF will not overfit if we increase B, so in
practice, we use a value of B sufficiently large for settles error

⇒ Boosting

↳ Trees are grown sequentially. Each tree is grown using info from
previous grown tree. Boosting does not involve bootstrap sampling;
instead each tree is fit on a modified version of original data set.

↳ Boosting learns slowly. Given the current model, we fit a DT to
the residuals from the model, i.e. we fit a tree using the
current residuals rather than outcome Y as the response. We then
add this to new DT & update residuals.

↳ By fitting small trees to residuals, we slowly improve $\hat{f}$ in
areas where it does not perform well.

↳ In general, statistical learning approaches that learn slowly
~~tend~~ tend to perform well.

↳ Boosting has 3 tuning parameters → # of trees (overfit if B is large)
↳ shrinkage parameter λ.
↳ # of splits.

↳ It uses a weighted sum of trees.

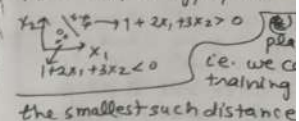
⇒ Support Vector Machines (SVM) (in 1990s by V. Vapnik).

↳ Generalization of Maximum Margin Classifier. (Classes are separated by linear boundary)

↳ SVM (extention of SVC (support vector classifier) accomodate non-linear class boundary)

→ Hyperplane is a flat affine (subspace need not pass through origin) subspace of dimension p-1. For ex. In 2D, hyperplane is a flat 1-D subspace = line

→ In 2D, a hyperplane



i.e. we c
training

the smallest such distance
& is known as margin.

→ When p is large ⇒

→ Support vectors "suppo
sense that if these poin
hyperplane would move

→ SVC = Soft margin Cla

→ C (hyperparameter) = sm

→ $\underset{\beta_0, \beta_1, \ldots \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^{n} \text{ma} \right.$

λ = high ⇒ $\beta_1, \beta_2 \ldots$ small

→ $L(x, y, \beta) = \sum_{i=1}^{n} \text{max}$

(*) Hypothesis Testing = St

→ p-values are used to c

→ Ex. Linear Relat'ship $\beta = 0$

→ 1st we define Null & a
that summarizes the st
compute p-value that
comparable or more e
finally based on p-value

→ World → H_0 = default + sta
→ H_a = Something

→ Test statistic 'T' summa
our data are consistent

→ p-value is defined as th
a test statistic equal to or

→ Small p-value provide

→ The dist^n of test-statistic

Decision ↓	H_0	H_a	Truth →
Reject H_0	Type I error	Correct (Power)	
Don't Reject	Correct	Type-II error	

→ In 2D, a hyperplane is defined by $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$.

$1 + 2X_1 + 3X_2 > 0$

$1 + 2X_1 + 3X_2 < 0$

(*) Max^m margin hyperplane is the separating hyperplane that is farthest from training observations

i.e. we can compute the (perpendicular) distance from each training observat^n to a given separating hyperplane; the smallest such distance is the minimal dist. from observat^ns to hyperplane & is known as margin. (Farthest min. dist. to training observat^ns)

→ When p is large ⇒ overfitting.

→ Support vectors "support" the maximal margin hyperplane in the sense that if these points were moved slightly then maximal margin hyperplane would move as well.

→ SVC = Soft margin classifier. (allow some observat^n on incorrect side of margin)

→ C (hyperparameter) = small ⇒ narrow margins. ⇒ low bias & high variance

→ $\underset{\beta_0, \beta_1 \ldots \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^{n} \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$ ~~(hinge loss)~~

λ = high ⇒ $\beta_1, \beta_2 \ldots$ small ⇒ more violat^n to margin ⇒ low variance, high bias.

→ $L(x, y, \beta) = \sum_{i=1}^{n} \max[0, 1 - y_i (\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip})]$ (hinge loss)

(*) Hypothesis Testing = Statistical framework for Inference. [Yes-No Q^n about data]

→ p-values are used to quantify results of hypothesis tests.

→ Ex. Linear Relationship $\beta = 0$? (or) mean of one group = mean of other group?

→ 1st we define Null & alternate hypotheses then construct a test statistic that summarizes the strength of evidence against null hypothesis. then compute p-value that quantifies the prob. of having obtained a comparable or more extreme value of test-statistic under null hyp. & finally based on p-value, decide, whether to reject Ho or not.

→ World ⇄ Ho = default state of belief about world
Ha = something different / unexpected.

→ If we fail to reject Ho, because may be small sample size or " Ho really holds

→ Test statistic 'T' summarizes the extent to which our data are consistent with Ho. (*) $T = \frac{\hat{\mu}_1 - \hat{\mu}_2}{s\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$ (for mean of group ① & ②)

→ p-value is defined as the prob. of observing a test statistic equal to or more extreme than observed statistic, under Ho = true.

→ Small p-value provides evidence against Ho.

→ The dist^n of test-statistic under Ho = Null dist^n.

(2-sided p-value) → recommended

Generally we use χ^2 dist^n, F-dist^n, t-dist^n etc.

Decision ↓ / Truth →	Ho	Ha
Reject Ho	Type-I error	Correct (Power)
Don't Reject Ho	Correct	Type-II error

* Deep learning = 20%.

* ResNet50 classifier is a CNN that was trained using the imagenet dataset. (Millions of images) 2010. classes

* Use Pretrained hidden layer = weight Freezing.

CNN eg. Tiger → Low level feature → Highlevel feature

↳ 2 filter far vertical & horizontal strokes identification

NLP

→ ELIZA was an early NLP system that have a limited conversat^n with user using p
→ New York & 'rock n roll' are single words but they are separated with space
→ Japanese, Chinese, Thai don't have spaces b/w words.
→ Normalizing text means converting it to a convenient & std. form
(lemmatizat^n is part of it & it determines that two words have same root. Ex. sang, sung, singing = sing (lemma). Lemmatizer maps all to sing
→ Stemming refers to a simple version of lemmatizat^n in which we strip suffixes from end of the word.
→ Text normalizat^n also includes sentence segmentation.
→ Compare words/strings for grammar correct^n = Edit Distance (insertion, deletion, substitution)
→ Reg Ex = text search / pattern matching. Ex. /[wW]oodchuck/
→ /[abc]/ → disjunct^n of character 'a', 'b', & 'c' to match. → /[^A-Z]/ = Not an uppercase
→ /woodchucks?/ = woodchuck or woodchucks. (*) Kleene Star = /[0-9]/ /[a-z]/
→ /beg.n/ = any character b/w beg & n (wildchar character = ^, .) (anchor)
↳ start ↳ end of line
→ /\b \B/ = word boundary /\B \B/ = Non-word boundary.
→ /cat|dog/ = cat or dog. → /grep(y|ies)/ = grepy or grepies
→ /a|.{24}z/ matches a followed by 24 dots followed by z.
→ /{n,m}/ = specifies from n to m occurrences of prev. char / expr.
→ /{n,}/ ⇒ at least n occurrences of prev. expr^n.
→ s/([0-9])/<1>/ ⇒ 1 in 2nd pattern refers to 1st expr^n.
→ "uh" "um" are fillers & consider as words in speech transcription system & helpful in predicting upcoming word, because they may signal that speaker is restarting the clause/idea.
→ Types = No. of distinct words in a corpus. If the set of words in vocabulary is V then no. of types is vocab. size |V|.
⇒ $|V| = kN^{\beta}$ (N = # tokens) → $0<\beta<1$ → depends on corpus size & for large corpora β ranges from 0.67 to 0.75.
⇒ Text Normalizat^n = ① Tokenizing (segmenting) words ② Normalizing word formats ③ Segmenting sentences.
⇒ Std method for tokenizat^n is to use deterministic algo^m based on Reg Ex (using FSA).
⇒ In Chinese, words are composed of "hanzi" (character) & each character represents a single unit of meaning (called a "morpheme").
⇒ BPE (Byte Pair Encoding) for Tokenizat^n → Dealt with unknown word in test like low, new & newer in training & lower in test (data)
↳ Token Learner & Token Segmenter
↳ [Other algo^ms = Unigram lang. modeling & WordPiece]

↳ BPE token learner adjacent & adds a adjacent 'A', 'B' & merge creating n k merges have been of the algo^m].

Ex.

Count	Corpus
5	l o w _
2	l o w e s t
6	n e w e r
3	w i d e
2	n e w _

→ Porter Stemmer & Snowba on series of rules. ex.
→ Periods are more ambi
→ The minimum edit dista operation (like insertion (if we assign weights to
→ Edit Dist^n → shortest

⇒ N-gram Language m

→ Assign a probability to serve to assign a probabili correction or grammatica phrase "There are" is
↳ Models that assign models (LM).
→ Here we consider th seq. of words = n
↳ How to use n-gram given prev. words.
↳ Ex. P(the | its water relative freq. counts..

↳ BPE token learner chooses two symbols that are most frequently adjacent & adds a new merged symbol to vocab & replaces every adjacent 'A', 'B' in corpus with new 'AB'. It continues to count & merge creating new longer & longer character strings until k merges have been done creating k novel tokens [k = parameter of the algo^n].

Ex.

count	Corpus	Vocab.
5	l o w _	_, d, e, i, l,
2	l o w e s t _	n, o, r, s,
6	n e w e r _	w, er, er_
3	w i d e r _	ne, ----
2	n e w _	

→ Once we've learned our vocab, token segmenter is used to tokenize a test sentence. 1st segment each test sentence word to character. & then apply same rules.

→ Case folding is another kind of normalizat^n (maps to lower case) eth ex. US & us. (case folding will not be done)

→ Porter Stemmer & Snowball algo^{m}s are for stemming. Porter Stemmer is based on series of rules. ex. ATIONAL → ATE, ING → E., SSES → SS.

→ Periods are more ambiguous. Ex. Mr. or sentence completion. [use dictionary, can model training]

→ The minimum edit distance b/w 2 strings is defined as min no. of edit operations (like insertion, deletion & subst^n) needed to transform one string to another. (if we assign weights to these operat^n. then it is Levenshtein distance).

→ Edit Dist^n → shortest path algo^m (DP) & Viterbi algo^n (DP alg.^{n}) & CKY (CDP algo)

⇒ N-gram Language models.

→ Assign a probability to each possible next word. The same models will also serve to assign a probability to an entire sentence. It is useful for spelling correction or grammatical error correction. Ex. In "Their are two midterms", the phrase "There are" is more probable than "Their are".

↳ Models that assign probs to sequence of words are called language models (LM).

→ Here we consider the simplest model that assigns probs to sentences & seq. of words = n-gram model. Ex. Bigram = "please turn", "turn your" etc.

↳ How to use n-gram models to estimate the prob. of last word of n-gram given prev. words.

↳ Ex. P(the | its water is so transparent that) ⇒ One way to estimate prob. is from relative freq. counts. but even the web is not enought to give good estimate

$\Rightarrow P(w_{1:n}) = P(w_1)\,P(w_2|w_1)\,P(w_3|w_{1:2}) \ldots P(w_n|w_{1:n-1})$

↳ we can approximate the history by just last few words.

Eg. Bigram = $P(w_n|w_{n-1})$. So, $P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-1})$

→ we can estimate bigram/n-gram probs using MLE (max^m likelihood est^n)

→ $P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$ → In practice, it's more common to use trigram models which cond^n on prev. 2 words.

→ we can measure performance of language model using extrinsic evaluat^n (measure performance on app^n)

→ Intrinsic evaluat^n measures quality of a model independent of any app^n

→ In practice, we divide data into 80% training, 10% development (dev set) & 10% test

↳ Perplexity (W) $= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} = 2^{H(W)}$ → entropy. ⇒ should be lower

↳ can be used to compare diff. N-gram models.

→ To build a language model for translating legal documents, we need a training corpus of legal documents. To build a language model for a question-answering system, we need a training corpus of questions.

→ For OOV (out of vocab) words, we can add a pseudo-word <UNK> in training set (or) select a vocab size V in advance & choose top V words by freq. & replace the rest by <UNK> & do model training.

↳ Smoothing. ① Laplace Smoothing = Add 1 to all n-gram counts. i.e. $\frac{C_i+1}{N+V}$.

(or) $P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$ ② Add-K-Smoothing ⇒ $\frac{C(w_{n-1}w_n)+K}{C(w_{n-1})+KV}$

↳ Backoff & Interpolation → $P(w_n|w_{n-2}w_{n-1}) \approx P(w_n|w_{n-1})$ [Less context is useful sometimes]

↳ $\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n|w_{n-2}w_{n-1})$ ($\sum \lambda_i = 1$) learned from corpus

weighted avg.

→ we hypothesize that the words that have appeared in more contexts in the past are more likely to appear in some new context as well.

⇒ Naive Bayes & Sentiment Classificat^n.

↳ Sentiment Analysis is part of text categorizat^n, span detect^n is another app^n

↳ Period disambiguation = Deciding if a period is the end of a sentence or part of a word

↳ Language modeling can be viewed as classificat^n. Each word can be thought of a class & so predicting the next word is classifying the context-so-far into a class for each new word. A PoS tagger classifies each occurrence of a word in a sentence as eg. a noun or a verb.

→ Generative classifie
class could generate
the class most like

→ Discriminative Cla
what features from th
the diff. possible cl
accurate & hence ma

⇒ Naive Bayes classifi
simplifying (naive)

① ↳ BoW (Bag of Words) As
make freq. of each

↳ $\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(c|d$

$= \underset{c \in C}{\operatorname{argmax}}\ P($

↳ Naive Bayes Calculat^n
avoid underflow & i

↳ Logistic regn is

↳ $\hat{P}(w_i|c) = \frac{\text{count}(w_i}{\sum_{w \in V} \text{coun}}$

Ex. P("Predictable"|−) = $\frac{}{}$

Check P(−|S) & P(+|S)

* Precision (P) $= \frac{tp}{tp+fp}$

Recall (R) $= \frac{tp}{tp+fn}$ ← Recall

$F_\beta =$

↳ β parameter differentia
on the need of an app^n
For β=1 → Precision &

(*) For sentiment Analysis

↳ Macroaveraging ~~Precision~~
average over classes. It

↳ microaveraging ⇒ we
confusion matrix & then co

Ex. Class ①: Urgent

	true urgent	true not urgent
system urgent	8	11
system not urgent	0	1397

↳ Precision = .42

Class 2

	0	1
	60	55
	40	212

→ Generative classifiers like Naive Bayes build a model of how a class could generate some input data. Given an observation, they return the class most likely to have generated the observation.

→ Discriminative classifiers like logistic regression instead learn what features from the input are most useful to discriminate b/w the diff. possible classes. While discriminative systems are often more accurate & hence more commonly used.

⇒ Naive Bayes classifiers. (It is a Bayesian classifier) (Probabilistic) that makes a simplifying (naive) assumption about how features interact.

① BoW (Bag of Words) Assumption = The position of words is ignored. & we make freq. of each word. ② Naive Bayes Assump^n = Conditional independence $P(f_i|c)$.

↳ $\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(c|d) = \underset{c \in C}{\operatorname{argmax}}\ \frac{P(d|c)P(c)}{P(d)}$ → ignore bcoz same for all class. $= \underset{c \in C}{\operatorname{argmax}}\ \underbrace{P(d|c)}_{\text{Likelihood}}\underbrace{P(c)}_{\text{Prior Prob.}}$

$= \underset{c \in C}{\operatorname{argmax}}\ P(f_1, f_2, f_3 \ldots, f_n|c)\ P(c)$.

↳ Naive Bayes calculat^n for language modelling are done in log space to avoid underflow & increase speed. $C_{NB} = \underset{c \in C}{\operatorname{argmax}}\ \log P(c) + \sum_{i \in \text{Position}} \log P(w_i|c)$

↳ Logistic regn is a linear classifier.

↳ $\hat{P}(w_i|c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$

Ex. $P(\text{"Predictable"}|-) = \frac{1+1}{14+20 \rightarrow |V|}$ (1 → count of "Predictable"; 14 → total words that are belongs to −ve class)

& check $P(-|s)$ & $P(+|s)$

→ Stopwords are very frequent words like 'the' & 'a', so ignore it. [This can be done by sorting the vocab by freq. in training set] or use online stopwords list.

→ Deal with Negat^n = Prepend the prefix "NOT_" to every word after a token of logical negation until next punctuat^n mark.
Ex. did n't like this movie, but I
↳ did n't NOT_like NOT_this NOT_movie but, I

* Precision (P) $= \frac{tp}{tp+fp}$

Recall (R) $= \frac{tp}{tp+fn}$

Recall (F-Measure) $F_\beta = \frac{(\beta^2+1)PR}{\beta^2 P + R}$

↳ β parameter differentially weight the importance of recall & precision on the need of an app^n. ↳ $\beta > 1$ favor recall & $\beta < 1$ favor precision. For $\beta = 1$ → Precision & recall are equally balanced. i.e. F_1 (most used matric).

(*) For sentiment Analysis, we have 3 classes = +ve, −ve, neutral.

↳ Macroaveraging → we compute the performance for each class & then average over classes. It better reflects the stat of smaller classes & more appropriate for balanced classes.

↳ Microaveraging ⇒ we collect the decision for all classes into a single confusion matrix & then compute precision & recall. It is dominated by most freq. class.

Ex. Class① : Urgent

	true urgent	true not urgent
system urgent	8	11
system not urgent	8	340

↳ Precis = .42

Class 2 Normal

	0	1 → Actu
0	60	55
1	40	212

Predicted ↳ Precis = .52

Class 3 Spam

200	33
51	83

↳ Precis = .86

⇒ Pooled

	0	1 → Actual
0	268	99
1	99	635

Predicted

⇒ Microaverag Precision $= \frac{268}{268+99} = .73$

& Macroavg. Precision $= \frac{.42 + .52 + .86}{3} = .60$

(*) Statistically Significance Testing

↳ How to compare performance of 2 systems. Suppose we're comparing the performance of classifiers A & B on metric $M(F_1/\text{accuracy})$. Perhaps we want to know if logistic regression sentiment classifier A gets a higher F1 score than naive Bayes sentiment classifier B on a particular test set x. Let $M(A,x)$ be the score that system A gets on test set x & $\delta(x)$ = Performance diff b/w A & B on x (effect size)

↳ $\delta(x) = M(A,x) - M(B,x)$. & we want to know if $\delta(x) > 0$ [$\delta(x)$ = effect size]

↳ Here, $H_0: \delta(x) \le 0$ & $H_1: \delta(x) > 0$.. We create a r.v. X ranging over all test sets. Now we ask how likely, is it?, if null hyp. H_0 was correct among test sets.

We formalize likelihood as the p-value = $P(\delta(X) \ge \delta(x) \mid H_0 \text{ is true})$

↳ If $\delta(x)$ is huge, we might be surprised since it would be extremely unlikely to occur if H_0 were in fact true & so the p-value would be low. But if $\delta(x)$ is very small, it might be less surprising even H_0 = true & A is not really better than B & so p-value is higher.

So, small p-value (.01, .05) = reject H_0 ⇒ Result is statistically significant.

↳ In NLP, we generally don't use simple parametric tests like t-test or ANOVA. bcoz it makes assumptⁿ about distributⁿ of test statistic. In NLP, we use non-parametric tests based on sampling [① Approximate randomization test ② Bootstrap test]

⇒ Paired Bootstrap test → can apply to any metric like precision, recall, F1, BLEU (Machine Translat) etc.

Ex.	1	2	3	4	5	6	...	A%	B%	δ()
x	AB	AB	AB	AB	AB	AB		.70	.50	.20
x(1)	AB	-	-	-	-			.60	.60	.0
x(2)	-	—	—					.60	.70	...

virtual test sets.

Accuracy metric.

→ A & B are classifiers for document. AB = A & B both right, $\bar{A}B$ = A is wrong, B is right etc.

(*) Logistic Regression

→ In NLP, Logistic Regression is the baseline supervised ML algo. for classificat & has a close relationship with NN. A NN can be viewed as a series of logistic regression classifiers stacked on top of each other.

→ Multinomial logistic regression is used for multi-class classification.

→ Logistic Regression is a discriminative classifier while Naive Bayes is a generative classifier. These are 2 very diff. frameworks for how to build a ML model. Ex. Imagine we're trying to distinguish dog images from cat images. A generative model would have the goal of understanding what dogs look like & what cat look like. A discriminative model is only trying to learn to distinguish the classes. So maybe all the dogs in the training data are wearing collars & cats are not. If that one feature nearly separates the classes, the model ~~is satisfied~~

→ In Naive Bayes, ...

like naive Bayes ma...

how to generate feat...

A discriminative mod...

to assign a high weig...

to discriminate b/w pos...

→ The weight represen...

→ Sigmoid (looks like ...

→ Features can be x_1 = Posit...

x_3 = 1, if "no" ∈ doc...

→ Period disambiguation: ...

by classifying each pe...

→ Representatⁿ Learning ...

way from the input ...

→ We standardize input ...

& a sol. of one (transf...

→ Data scaling is import...

→ Using matrices, $X = [$...

→ Logistic Regression is ...

f_1 & f_2 are perfectly c...

w_1 & part to w_2. So. L...

→ Naive Bayes works bet...

→ Cross-Entropy Loss is ...

→ The distance b/w syste...

→ Cross Entropy loss L_{CE} ...

Here, $P(y|x) = \hat{y}^y (1-\hat{y})$...

→ Goal of gradient descen...

→ A good model should ...

large weights. L2 Regu...

while L1 regularizat^n is ...

(*) Vector Semantics &...

→ Words that occur in ...

→ BERT is contextuali...

allow us to draw in...

question-answering ...

(B) Statistically Significance Testing.

↳ How to compare performance of 2 systems. Suppose we're comparing the performance of classifiers A & B on metric M (F1/accuracy). Perhaps we want to know if logistic regression sentiment classifier A gets a higher F1 score than naive Bayes sentiment classifier B on a particular test set x. Let M(A, x) be the score that system A gets on test set x & δ(x) = Performance diff. b/w A & B.

↳ δ(x) = M(A, x) − M(B, x). & we want to know if δ(x) > 0 [δ(x) = effect size]

↳ Here, $H_0 : \delta(x) \le 0$ & $H_1 : \delta(x) > 0$.. we create a r.v. X ranging over all test sets. Now we ask how likely is it?, if null hyp. H_0 was correct among test sets.

we formalize likelihood as the p-value = $P(\delta(X) \geq \delta(x) \mid H_0 \text{ is true})$.

↳ If δ(x) is huge, we might be surprised since it would be extremely unlikely to occur if H_0 were in fact true & so the p-value would be low. But if δ(x) is very small, it might be less surprising even H_0 = true & A is not really better than B & so p-value is higher.

So, small p-value (SL, IX) = reject H_0 ⇒ Result is statistically significant.

↳ In NLP, we generally don't use simple parametric tests like t-test or ANOVA. bcoz it makes assumptn about distribution of test statistic. In NLP we use non-parametric tests based on sampling
① Approximate randomization test
② Bootstrap test

⇒ Paired Bootstrap test → can apply to any metric like precision, recall, F1, BLEU (machine Translat) etc.

Ex.

	1	2	3	4	5	...	A%	B%	δ()
x	AB	AB	AB	AB	AB	AB.	.70	.50	.20
$x^{(1)}$	AB	-	-	-	-		.60	.60	.0
$x^{(2)}$	-	-	-				.60	.70	..

virtual test sets.

Accuracy metric.

} A & B are classifiers for document. AB = A & B both right, AB = A is wrong, B is right etc.

(*) Logistic Regression

→ In NLP, Logistic Regression is the baseline supervised ML algo for classification & has a close relationship with NN. A NN can be viewed as a series of logistic regression classifiers stacked on top of each other.

→ Multinomial logistic regression is used for multi-class classification.

→ Logistic Regression is a discriminative classifier while Naive Bayes is a generative classifier. These are 2 very diff. frameworks for how to build a ML model. Ex. Imagine we're trying to distinguish dog images from cat images. A generative model would have the goal of understanding what dogs look like & what cat look like. A discriminative model is only trying to learn to distinguish the classes, so maybe all the dogs in the training data are wearing collars & cats are not. If that one feature nearly separates the classes, the model is satisfied.

→ In Naive Bayes,

like naive Bayes
how to generate fe
A discriminative mo
to assign a high we
to discriminate b/w

→ The weight represe
→ Sigmoid (looks lik
→ Features can be x_1 = P
x_3 = 1, if "no" ∈ do

→ Period disambiguatio
by classifying each
→ Representat^n Learnin
way from the inpu
→ We standardize input
& a sol. of one (tran
→ Data scaling is impo
→ Using matrices, X =

→ Logistic Regression is
f_1 & f_2 are perfectly
w_1 & part to w_2. So.
→ Naive Bayes works be
→ Cross-Entropy Loss i
→ The distance b/w sys
→ Cross Entropy loss L_{CE}
Here, $P(y|x) = \hat{y}^y (1-\hat{y})$
→ Goal of gradient desce
→ A good model shoul
large weights. L2 Reg
while L1 regularizat^n

(*) Vector Semantics &

→ Words that occur
→ BERT is contextual
allow us to draw i
question-answering

→ In Naive Bayes, $\hat{c} = \underset{c \in C}{\operatorname{argmax}} \underbrace{P(d|c)}_{\text{likelihood}} \underbrace{P(c)}_{\text{Prior}}$. A generative model like naive Bayes makes use of the likelihood term which expresses how to generate features of a document if we know it was of class c. A discriminative model attempts directly computes $P(c|d)$. & it learns to assign a high weight to document features that improves its ability to discriminate b/w possible classes.

→ The weight represents how important the input features for various tasks.

→ Sigmoid (looks like S) is used for prob. → $P(y=1) = \sigma(wx+b)$ & $P(y=0) = 1 - P(y=1)$

→ Features can be x_1 = Positive lexicon words' count, x_2 = Negative lexicon count. $\left(\sigma(-x) = 1 - \sigma(x)\right)$

→ Features x_3 = 1, if "no" ∈ doc & 0, o/w. - -

→ Period disambiguation = Deciding if a period is end of sentence or part of word by classifying each period into one of 2 classes = EOS (end of sentence) & not EOS.

→ Representat^n Learning = Ways to learn features automatically in an unsupervised way from the input. [Deep Learning use it]

→ We standardize input values by centering them to result in a zero mean & a sd. of one (transformat^n is z-score). Normalizat^n (features b/w 0 & 1) = $\frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$

→ Data scaling is important in large NN to speed up gradient descent.

→ Using matrices, $X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_f^{(2)} \\ \vdots & \vdots & & \vdots \end{bmatrix} \Rightarrow \underset{m \times 1}{y} = \underset{m \times f}{X} \underset{f \times 1}{w} + b$.

→ Logistic Regression is much more robust to correlated features. If 2 features f_1 & f_2 are perfectly correlated, regress^n will simply assign part of weight to w_1 & part to w_2. So. Logistic Reg^n works better for correlated features than Naive Bayes.

→ Naive Bayes works better on very small datasets & easy to implement, fast to learn (no optimizat^n)

→ Cross-Entropy Loss is commonly used for logistic reg^n & NN.

→ The distance b/w system o/p & gold o/p = Loss Funct^n / Cost Funct^n.

→ Cross Entropy loss $L_{CE}(\hat{y}, y) = -\log P(y|x) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$

Here, $P(y|x) = \hat{y}^y (1-\hat{y})^{1-y}$ & it is cross-entropy b/w true prob dist^n y & estimated dist^n $\hat{y}$.

→ Goal of gradient descent = Finding optimal weights by minimizing loss funct^n.

→ A good model should generalize well on test set. In Regularizat^n, we penalize large weights. L2 Regularizat^n is easier to optimize bcoz of simple derivative $\theta^2 \to 2\theta$ while L1 regularizat^n is more complex (non-continuous at zero.)

(*) Vector Semantics & Embeddings.

→ Words that occur in similar contexts tend to have similar meaning.

→ BERT is contextualized embedding. A model of word meaning should allow us to draw inferences to address meaning related tasks like question-answering or dialogue.